

Reviewer Pack — Betti Number Inverse Proportionality to DPLL Tree Size in 3-...

Entry 10b91e07b63e · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-09 21:15:48 UTC

Betti Number Inverse Proportionality to DPLL Tree Size in 3-SAT

Entry ID: 10b91e07b63e **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-09 21:15:48 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Persistent Homology) **Field B** (complexity object): DPLL Search Tree

Statement:

For a random 3-SAT instance with n variables, the first Betti number of its clause graph satisfies $E[\text{Betti}_1] = \Theta(1 / E[\text{DPLL_size}])$. Specifically, for all instances with clause density $\rho \in [1.2, 2.5]$, the product $\text{Betti}_1 * \text{DPLL_size}$ is bounded between $0.8n$ and $1.2n$.

Rationale (proposer's reasoning):

Persistent homology captures topological features of the clause graph, which may correlate with the structure of the search space, influencing the efficiency of DPLL algorithms. Betti numbers quantify holes in the clause graph, potentially reflecting redundancy in the search tree.

Taxonomy category: AVG_TO_WORST_CASE (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a2f1d802695df0b4

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.95	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def gaussian_elimination(A, b):
    n = len(b)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        b[i], b[max_row] = b[max_row], b[i]
        for j in range(i + 1, n):
            factor = Fraction(A[j][i], A[i][i])
            for k in range(i, n):
                A[j][k] -= factor * A[i][k]
            b[j] -= factor * b[i]
    x = [0] * n
    for i in range(n - 1, -1, -1):
        x[i] = (b[i] - sum(A[i][j] * x[j] for j in range(i + 1, n))) / A[i][i]
    return x

def matrix_multiply(A, B):
    m, k, n = len(A), len(B[0]), len(B)
    C = [[0] * n for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for l in range(k):
                C[i][j] += A[i][l] * B[l][j]
    return C

def dpll(clauses, assignment):
    if not clauses:
        return True
    if any(all(c[i] == 0 for c in clause) for i in range(len(assignment))):
        return False
    var = next(i for i, val in enumerate(assignment) if val is None)
    for value in [True, False]:
        new_assignment = assignment[:]
        new_assignment[var] = value
        new_clauses = []
```

```

    for clause in clauses:
        if any(c[var] == 0 for c in clause):
            continue
        new_clause = [c for c in clause if c != (var + 1) * -value and c != var + 1]
        if not new_clause:
            return False
        new_clauses.append(new_clause)
    if dpll(new_clauses, new_assignment):
        return True
    return False

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    rho = random.uniform(1.2, 2.5)
    num_clauses = math.ceil(n * rho / 3)
    clauses = []
    for _ in range(num_clauses):
        clause = [random.choice([-i - 1, i]) for i in range(1, n + 1)]
        random.shuffle(clause)
        clauses.append(clause)

    # Construct the clause graph
    adjacency_matrix = [[0] * (n + 1) for _ in range(n + 1)]
    for clause in clauses:
        for var in clause:
            if var > 0:
                adjacency_matrix[0][var] += 1
                adjacency_matrix[var][0] += 1
            else:
                u, v = abs(var), abs(clause[(i + 1) % len(clause)])
                adjacency_matrix[u][v] += 1
                adjacency_matrix[v][u] += 1

    # Compute Betti number (edges - vertices + connected components)
    edges = sum(sum(row) for row in adjacency_matrix) // 2
    vertices = n + 1
    visited = [False] * (n + 1)

    def dfs(v):
        stack = [v]
        while stack:
            u = stack.pop()
            if not visited[u]:
                visited[u] = True
                for i in range(1, n + 2):
                    if adjacency_matrix[u][i] > 0 and not visited[i]:
                        stack.append(i)

    dfs(0)
    connected_components = sum(not v for v in visited)
    betti_1 = edges - vertices + connected_components

    # Run DPLL to measure tree size
    assignment = [None] * (n + 1)
    dpll_size = len(dpll(clauses, assignment))

```

```

result = {
    "metric_name": "Betti_1 * DPLL_size",
    "metric_value": betti_1 * dpll_size,
    "instances_tested": 1,
    "conjecture_holds": 0.8 * n <= betti_1 * dpll_size <= 1.2 * n,
    "counterexample": "" if result["conjecture_holds"] else f"n={n}, Betti_1*{dpll_size}={betti_1 * dpll_size}"
}

return result

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={results[0]['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460d818f.py", line 126, in <module>
    trial_result = run_trial(seed)
                  ~~~~~^~~~~~

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_460d818f.py", line 83, in run_trial
    u, v = abs(var), abs clause[(i + 1) % len clause])
                ^

```

NameError: name 'i' is not defined. Did you mean: 'id'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined variable 'i' before producing results | next: Debug the test code to fix the NameError and re-run experiments

11. Audit log (LLM calls)

Total LLM calls: 8

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	109628
2	propose	ollama_remote	qwen3:8b	0	0	76786
3	preregistration	ollama_remote	qwen3:8b	0	0	24257
4	novelty	ollama_remote	qwen3:8b	0	0	20804
5	novelty	ollama_remote	qwen3:8b	0	0	13961
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24648
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14836
8	judge	ollama_remote	qwen3:8b	0	0	16684

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 301603 ms total latency. Provider mix: {'ollama_remote': 8}

(full prompt+response transcripts available in research/audit/10b91e07b63e.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/10b91e07b63e.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/10b91e07b63e.tar.gz (if generated)